

GNU Debugger (GDB)

Sovellusten tietoturva ja hakkerointi



GNU Debugger

- GNU Debugger eli gdb on GNU-projektin ohjelma, jolla voi tarkastella ohjelman toimintaa lähemmin eli debuggeri.
- Käytetään komentoriviltä.
- Runsaasti erilaisia graafisen käyttöliittymän toteuttavia laajennuksia kuten Data Display Debugger (GNU DDD).
- Mahdollista käyttää GDB:tä etänä verkkoyhteyden yli, jolloin käyttöliittymä on työasemalla ja debuggattava ohjelma toisella koneella.
- Ohjelmalla pystyy debuggaamaan esimerkiksi useita eri ohjelmointi kieliä kuten Ada, C, C++, Objective-C, Free Pascal, Fortran ja Java-ohjelmia.
- Ohjelman kirjoitti alun perin Richard Stallman vuonna 1986. Esikuvana oli dbx-ohjelma, joka toimitettiin BSD-Unixin mukana.
- Ohjelman lähdekoodin muunnos C++11-version on saatu valmiiksi vuonna 2017.

Main.cpp

```
#include<iostream>

using namespace std;

long factorial(int n);

int main()
{
    int n(0);
    cin>>n;
    long val=factorial(n);
    cout<<val;
    cin.get();
    return 0;
}

long factorial(int n)
{
    long result(1);
    while(n-->0)
    {
        result*=n;
    }
    return result;
}
```

Ongelma tässä koodissa on factorial-funktion logiikassa. Tarkemmin sanottuna, while-silmukassa käytetty `n--` vähentää `n` arvoa ennen kuin sitä käytetään kertolaskussa. Tämä johtaa virheellisiin laskutoimituksiin, koska ensimmäinen kertolasku tehdään `n-1`:llä.

Esimerkiksi, jos `n` on 5, silmukka suorittaa kertolaskut muodossa `result *= 4 * 3 * 2 * 1 * 0`, mikä on virheellinen.

Korjatussa logiikassa pitäisi ensin kertoa nykyisellä `n`:n arvolla ja sitten vähentää sitä, jolloin varmistetaan, että jokainen arvo `n`:stä alaspäin arvoon 1 kerrotaan oikein.

How the code is
compiled

Koodin kääntäminen

```
g++ main.cpp -o main
```

- Ylläoleva komento kääntää koodin ajettavaan muotoon

```
-rwxr-xr-x 1 lisoant lisoant 71040 Oct 3 23:58 main  
-rw-r--r-- 1 lisoant lisoant 314 Oct 3 23:18 main.cpp  
-rwxr-xr-x 1 lisoant lisoant 88008 Oct 3 23:58 main-dbg
```

- Debuggausta varten main-dbg on käännetty eri tavalla

```
g++ main.cpp -g -Wall -Werror -o main
```

Koodin kääntäminen

```
g++ main.cpp -g -Wall -Werror -o main-dbg
```

- **-g**

- Lisää ohjelmaan debug-tietoja. Sen avulla voit suorittaa ohjelman debug-ohjelmassa ja seurata virheitä ja ohjelman suorittamista tarkemmin.

- **-Wall**

- Optio ottaa käyttöön kaikki yleiset varoitukset, jotka kääntäjä pystyy antamaan. Se auttaa tunnistamaan mahdollisia virheitä ja epäselvyyksiä koodissa.

- **-Werror**

- Optio muuntaa kaikki varoitukset virheiksi, eli jos koodissa on varoituksia, kääntäjä ei tee suoritettavaa ohjelmaa ennen kuin varoitukset on korjattu. Tämä auttaa pitämään koodin virheettömänä.

DEBUGGING

An illustration of a woman with long red hair, wearing a white shirt, sitting at a desk and typing on a keyboard. She is looking at a computer monitor displaying green code on a dark background. The word 'DEBUGGING' is written in large, bold, yellow letters across the center of the image. In the background, there is a window with blinds and a potted plant. On the desk, there is a red water bottle and a small black container.

Käyttö

```
gdb ./main-dbg
```

```
(lisoant@varvas)-[~/s]  
$ gdb ./main-dbg  
GNU gdb (Debian 15.1-1) 15.1  
Copyright (C) 2024 Free Software Foundation, Inc.  
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>  
This is free software: you are free to change and redistribute it.  
There is NO WARRANTY, to the extent permitted by law.  
Type "show copying" and "show warranty" for details.  
This GDB was configured as "aarch64-linux-gnu".  
Type "show configuration" for configuration details.  
For bug reporting instructions, please see:  
<https://www.gnu.org/software/gdb/bugs/>.  
Find the GDB manual and other documentation resources online at:  
  <http://www.gnu.org/software/gdb/documentation/>.
```

```
For help, type "help".  
Type "apropos word" to search for commands related to "word" ...  
Reading symbols from ./main-dbg...  
(gdb)
```

Käyttö – komennot lyhyesti

- Lähdekoodin näyttäminen – l tai list
- Keskeytyspisteen asettaminen – break <rivi nro>
 - break 17
- Keskeytyspisteen poisto – delete
- Näyttää asetetut keskeytyspisteet – info breakpoints
- Keskeytyspisteen passivointi ja aktivointi disable ja enable
- Ajaa ohjelmaa – run
- Poistu debuggerista – exit
- Siirtyy seuraavalle riville ja siirtyy myös funktioiden sisälle – step
- Siirtyy seuraavalle riville suorittamatta aliohjelmia tai funktioita – next
- Suorittaa nykyisen funktion loppuun ja palaa kutsuneeseen funktioon – finish
- Suorittaa koodia kunnes tietty rivi saavutetaan. - until

Running

```
# gdb <program> [core dump]
    Start GDB (with optional core dump).

# gdb --args <program> <args...>
    Start GDB and pass arguments

# gdb --pid <pid>
    Start GDB and attach to process.

set args <args...>
    Set arguments to pass to program to
    be debugged.

run
    Run the program to be debugged.

kill
    Kill the running program.
```

Breakpoints

```
break <where>
    Set a new breakpoint.

delete <breakpoint#>
    Remove a breakpoint.

clear
    Delete all breakpoints.

enable <breakpoint#>
    Enable a disabled breakpoint.

disable <breakpoint#>
    Disable a breakpoint.
```

Watchpoints

```
watch <where>
    Set a new watchpoint.

delete/enable/disable <watchpoint#>
    Like breakpoints.
```

<where>

```
function_name
    Break/watch the named function.

line_number
    Break/watch the line number in the cur-
    rent source file.

file:line_number
    Break/watch the line number in the
    named source file.
```

Conditions

```
break/watch <where> if <condition>
    Break/watch at the given location if the
    condition is met.
    Conditions may be almost any C ex-
    pression that evaluate to true or false.

condition <breakpoint#> <condition>
    Set/change the condition of an existing
    break- or watchpoint.
```

Examining the stack

```
backtrace
where
    Show call stack.

backtrace full
where full
    Show call stack, also print the local va-
    riables in each frame.

frame <frame#>
    Select the stack frame to operate on.
```

Stepping

```
step
    Go to next instruction (source line), di-
    ving into function.
```

```
next
    Go to next instruction (source line) but
    don't dive into functions.

finish
    Continue until the current function re-
    turns.

continue
    Continue normal execution.
```

Variables and memory

```
print/format <what>
    Print content of variable/memory locati-
    on/register.

display/format <what>
    Like „print“, but print the information
    after each stepping instruction.

undisplay <display#>
    Remove the „display“ with the given
    number.

enable display <display#>
disable display <display#>
    En- or disable the „display“ with the gi-
    ven number.

x/nfu <address>
    Print memory.
    n: How many units to print (default 1).
    f: Format character (like „print“).
    u: Unit.

    Unit is one of:

        b: Byte,
        h: Half-word (two bytes)
        w: Word (four bytes)
        g: Giant word (eight bytes)).
```

Format

<i>a</i>	Pointer.
<i>c</i>	Read as integer, print as character.
<i>d</i>	Integer, signed decimal.
<i>f</i>	Floating point number.
<i>o</i>	Integer, print as octal.
<i>s</i>	Try to treat as C string.
<i>t</i>	Integer, print as binary (<i>t</i> = „two“).
<i>u</i>	Integer, unsigned decimal.
<i>x</i>	Integer, print as hexadecimal.

<what>*expression*

Almost any C expression, including function calls (must be prefixed with a cast to tell GDB the return value type).

file_name::variable_name

Content of the variable defined in the named file (static variables).

function::variable_name

Content of the variable defined in the named function (if on the stack).

{*type*}*address*

Content at *address*, interpreted as being of the C type *type*.

\$register

Content of named register. Interesting registers are \$esp (stack pointer), \$ebp (frame pointer) and \$eip (instruction pointer).

Threadsthread <*thread#*>

Chose thread to operate on.

Manipulating the programset var <*variable_name*>=<*value*>

Change the content of a variable to the given value.

return <*expression*>

Force the current function to return immediately, passing the given value.

Sourcesdirectory <*directory*>

Add *directory* to the list of directories that is searched for sources.

list

list <*filename*>:<*function*>list <*filename*>:<*line_number*>list <*first*>,<*last*>

Shows the current or given source context. The *filename* may be omitted. If *last* is omitted the context starting at *start* is printed instead of centered around it.

set listsize <*count*>

Set how many lines to show in „list“.

Signalshandle <*signal*> <*options*>

Set how to handle signals. Options are:

(no)print: (Don't) print a message when signals occurs.

(no)stop: (Don't) stop the program when signals occurs.

(no)pass: (Don't) pass the signal to the program.

Informations

disassemble

disassemble <*where*>

Disassemble the current function or given location.

info args

Print the arguments to the function of the current stack frame.

info breakpoints

Print informations about the break- and watchpoints.

info display

Print informations about the „displays“.

info locals

Print the local variables in the currently selected stack frame.

info sharedlibrary

List loaded shared libraries.

info signals

List all signals and how they are currently handled.

info threads

List all threads.

show directories

Print all directories in which GDB searches for source files.

show listsize

Print how many are shown in the „list“ command.

whatis *variable_name*

Print type of named variable.

Esimerkki

```
#include<iostream>

using namespace std;

long factorial(int n);

int main()
{
    int n(0);
    cin>>n;
    long val=factorial(n);
    cout<<val;
    cin.get();
    return 0;
}

long factorial(int n)
{
    long result(1);
    while(n-->0)
    {
        result*=n;
    }
    return result;
}
```

- Käännetään
 - g++ main.cpp -g -Wall -Werror -o main-dbg
- Käynnistetään debuggeri
 - gdb ./main-dbg
 - list
 - list
 - break 11
 - long val=factorial(n);
 - run
 - step
 - list
 - watch n
 - watch result
 - print val
 - next
 - continue
 - quit

Huomioitavaa

- <https://www.sourceware.org/gdb/current/onlinedocs/gdb.html/>
- <https://www.sourceware.org/gdb/current/onlinedocs/gdb.html/Threads.html>
- <https://developers.redhat.com/articles/the-gdb-developers-gnu-debugger-tutorial-part-1-getting-started-with-the-debugger>
- https://www.seas.upenn.edu/~ese5320/fall2021/handouts/hw1/gdb_tutorial.html
- <https://www.cprogramming.com/gdb.html>